- *Synchronous unit testing*—An actor instance is not directly accessible. To solve this problem, the test kit provides a `BehaviorTestKit` you can use to access the effects resulting from what is executed in the actor instance. Another tool is the `TestInbox`, which you can use to instantiate an `ActorRef` that serves as a probe.
- *Asynchronous unit testing*—The `akka-testkit` module provides `ActorTestKit` and `TestProbe`, which are similar to their synchronous counterparts. However, they have important differences: they create complete `ActorSystem`s and are therefore indeterministic.
- *Multiple JVM and node testing*—Akka provides tools for testing clusters, but that is not covered in this book. You can learn more at http://mng.bz/Jg8a.

The option closest to running your code in production is the asynchronous style, which is tested using the `ActorTestKit` class. In this book, the focus is on asynchronous testing because it can reveal problems in the code that would not be noticed in a stubbed-sync environment. This is usually the safest choice. It also probably won't surprise you that I recommend a classic unit-testing approach instead of mocking.

The classic approach tests that you get certain outputs from certain inputs to make sure your code works properly. This is called testing the *external behavior*. In mocking, on the other hand, you test the *internal behavior*: that is, the specific implementation. For example, you can test that your `insertClient` method calls the correct validation method and the DB method of your repository. A significant disadvantage of mocking is that if you now want to change the